

LaZagna: An Open-Source Framework for Flexible 3D FPGA Architectural Exploration

Ismael Youssef, Hang Yang, and Cong Hao

School of Electrical and Computer Engineering, Georgia Institute of Technology

Email: {ismael.youssef, hyang628, callie.hao}@gatech.edu

Abstract—While 3D IC technology has been extensively explored for ASICs, their application to FPGAs remains limited. Existing studies on 3D FPGAs are often constrained to fixed prototypes, narrow architectural templates, and simulation-only evaluations. In this work, we present LaZagna, the first open-source framework for automated, end-to-end 3D FPGA architecture generation and evaluation. LaZagna supports high-level architectural specification, synthesizable RTL generation, and bitstream production, enabling comprehensive validation of 3D FPGA designs beyond simulation. It significantly broadens the design space compared to prior work by introducing customizable vertical interconnect patterns, novel 3D switch block designs, and support for heterogeneous logic layers. The framework also incorporates practical design constraints such as inter-layer via density and vertical interconnect delay. We demonstrate the capabilities of LaZagna by generating synthesizable RTL that can be taken through full physical design flows for fabric generation, along with functionally correct bitstreams. Furthermore, we conduct five case studies that explore various architectural parameters and evaluate their impact on wirelength, critical path delay, and routing runtime. These studies showcase the framework’s scalability, flexibility, and effectiveness in guiding future 3D FPGA architectural and packaging decisions. LaZagna is fully open-source and available on GitHub¹

I. INTRODUCTION

Field-Programmable Gate Arrays (FPGAs) are widely adopted in modern computing systems due to their flexibility, reconfigurability, and inherent parallelism. As application demands continue to grow in performance and integration complexity, improving the architectural efficiency of FPGAs has become a critical research direction.

Three-dimensional (3D) integration presents a promising path forward by vertically stacking logic, routing, and memory resources. In the ASIC domain, 3D integration has been extensively explored, demonstrating substantial gains in performance, density, and energy efficiency [1]–[3]. In contrast, the application of 3D integration to FPGAs remains relatively underexplored, despite its potential to reduce interconnect delay, improve logic density, and support heterogeneous integration across layers.

Several prior works have proposed architectural enhancements for 3D FPGAs, such as separating configuration memory from logic [4]–[6], or enabling multi-layer routing and homogeneous stacking [7]–[10]. These studies report improvements in wirelength and critical path delay (CPD) compared to 2D counterparts. However, most proposals are limited to single or few designs, and rely solely on simulation-based evaluation without support for synthesizable RTL or bitstream generation. To advance the development of 3D FPGAs, there is a clear need for a powerful tool that can *automatically and extensively explore the 3D FPGA architectural design space*. Such a tool would not only facilitate the discovery of optimal 3D FPGA architectures, but also provide actionable insights into viable 3D packaging technologies.

However, supporting comprehensive 3D FPGA design space exploration (DSE) poses multiple challenges. First, the tool must support a wide variety of vertical interconnect types and densities, to enable future studies on the physical integration of cross-layer vias. Second, heterogeneous layer configurations must be considered, including flexible partitioning of configurable logic blocks (CLBs), DSPs, and BRAMs across layers, aligning with emerging trends in 3D ICs. Third, new types of 3D switch blocks (SBs) and/or connection blocks (CBs) must be designed and evaluated. Fourth, vertical delay must be taken into account when organizing the layer structure. Finally, to enable practical validation, the framework must generate not only simulated outputs but also synthesizable RTL and functionally correct bitstreams for real benchmarks.

To address these challenges, we present **LaZagna**, the first open-source framework for *automated 3D FPGA architecture generation and evaluation*. LaZagna enables rapid, end-to-end exploration of 3D FPGA designs, from high-level architectural specification to synthesizable RTL and bitstream generation. Unlike prior efforts focused on fixed designs or simulation-only flows, LaZagna offers comprehensive modeling of vertical connectivity, custom switch block design, heterogeneous logic layering, and seamless integration with open-source place-and-route and FPGA generation tools. We summarize our key contributions as follows:

- 1) **End-to-end and automated RTL and bitstream generation for 3D FPGAs:** LaZagna is the first framework to generate both synthesizable RTL and programming bitstreams for custom 3D FPGA fabrics, enabling evaluation beyond simulation. The tool is fully open-source to promote broad adoption and community-driven research.
- 2) **Full-spectrum and comprehensive 3D architecture exploration:** LaZagna supports scalable exploration of a wide range of 3D architectural parameters, including layer count, resource partitioning, inter-layer connectivity, and routing granularity. Beyond supporting known architectures, it introduces a significantly broader design space, including novel vertical interconnect patterns, flexible 3D switch block designs, and logic heterogeneity across layers.
- 3) **Evaluation and comprehensive case studies:** We demonstrate the capabilities of LaZagna by generating synthesizable RTL, performing full physical design flows, and producing functionally correct bitstreams for the generated 3D fabrics. To showcase the benefits of comprehensive DSE for future 3D FPGAs, we present five representative case studies that vary key architectural parameters and evaluate the designs in terms of wirelength, critical path delay, and routing runtime, demonstrating the framework’s utility for informed architectural decision-making.

II. PREVIOUS WORK AND LIMITATIONS

Related work falls into two main categories: (1) architectural studies on 3D FPGAs, and (2) tools for FPGA fabric and bitstream generation.

3D FPGA Architecture Studies. Various strategies have been proposed for vertical integration and layer organization in 3D FPGAs. We categorize these architectures into the following classes:

- **Homogeneous:** All layers share an identical layout, including logic blocks (e.g., CLBs, DSPs, BRAMs), routing components (e.g., SBs, CBs), and configuration memory.
- **Non-Logic Heterogeneous:** Logic blocks are placed on one layer, while other components such as SBs, CBs, or configuration memory are located on separate layers.
- **Logic Heterogeneous:** Each layer contains a different combination of logic resources (e.g., CLBs, DSPs, BRAMs), while maintaining full routing capabilities across layers.

Table I summarizes representative prior works on 3D FPGA architectures [4], [6]–[11]. In the homogeneous category, Amagasaki et al. [8] evaluated a 3D design (type 1) with inter-layer CLB outputs, which yielded no CPD improvement. Their follow-up work [9] introduced a four-layer architecture with two logic and two routing layers but observed a 9% degradation in CPD. Boutros et al. [10] proposed a homogeneous design with vertical interconnects restricted to CLB and hard IP outputs, demonstrating gains of 4% in wirelength and 3% in CPD. Le et al. [7] investigated scalability across 2 to 10 layers and reported significant CPD improvements, up to 61%, with a six-layer architecture.

¹<https://github.com/sharc-lab/LaZagna>

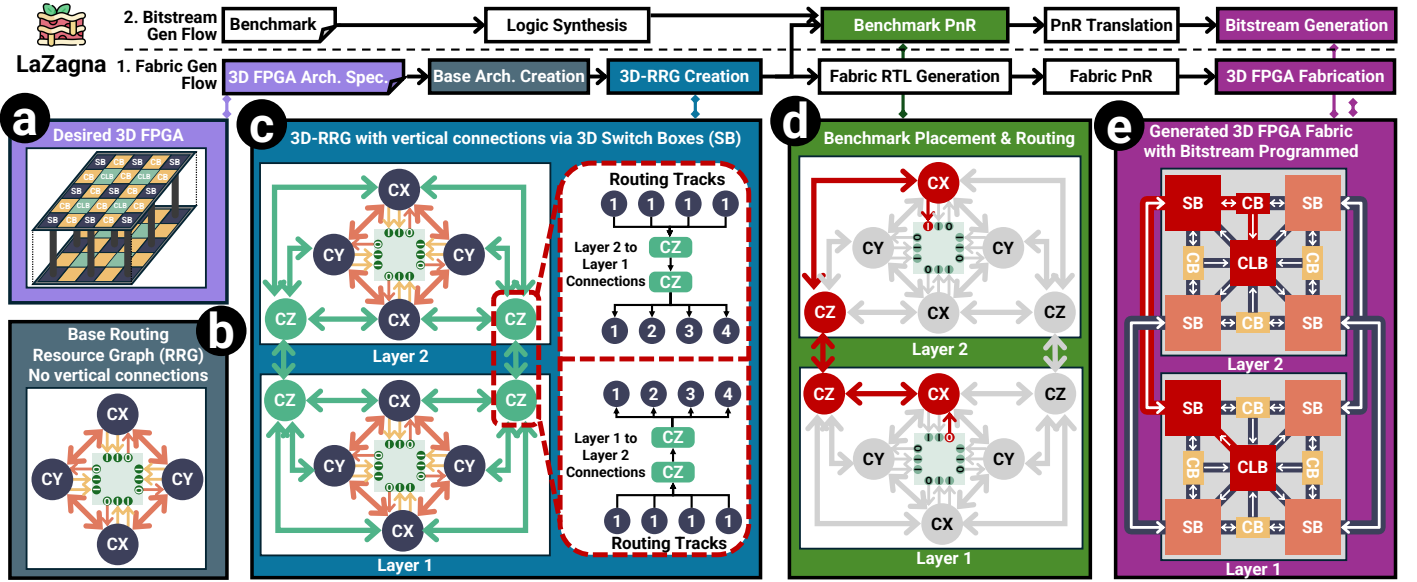


Fig. 1: LaZagna overview. It has two parallel flows: 1. 3D FPGA fabric RTL generation 2. benchmark bitstream generation.

FPGA Architecture	[8] type 1	[10]	[7]	[8] type 2	[9]	[4], [11], [6]	LaZagna Options
# of Layers	2	2	2-10	2	4	2	Arbitrary
Layer Type (Sec. III-A)	Homo	Homo	Homo	Non-Logic Hetero	Non-Logic Hetero	Non-Logic Hetero	Logic Hetero
Vertical Connection Type	3D CB	3D CB-O	3D SB	3D SB	3D CB-O	—	Self-defined (Table II)
Layers Configuration	—	—	—	1 Routing, 1 Logic	2 Routing, 2 Logic	1 Layer for Config. Memory	Self-defined (Table II)
LaZagna supports all features listed above and more (summarized in Table II), including FPGA architectural RTL and benchmark bitstream generation.							

TABLE I: LaZagna’s 3D FPGA modeling capabilities compared to previous works.

Non-logic heterogeneous designs have also demonstrated great performance potential. Amagasaki et al. [8] also explored a type 2 architecture that separates logic and routing across layers using 3D switch blocks, achieving a 5.3% CPD reduction over a 2D baseline. Turkyilmaz et al. [4] showed a 14% CPD improvement by separating configuration memory from logic. Lin et al. [5] isolated configuration memory onto a dedicated layer, resulting in an estimated $1.7\times$ CPD improvement. Waqar et al. [6] further advanced this idea using back-end-of-line integration to place configuration memory and SBs above CLBs, reporting CPD improvements ranging from 11% to 30%.

To the best of our knowledge, no prior work has discussed logic heterogeneous 3D FPGA architectures.

FPGA Tools. For open-source FPGA architecture exploration, the most widely used tool is Verilog-to-Routing (VTR) [12], which supports placement and routing for customizable 2D FPGA fabrics. However, VTR’s 3D capabilities are limited to two-layer homogeneous architectures, with inter-layer communication constrained to grid I/O pins. OpenFPGA [13] builds on VTR and supports RTL and bitstream generation for user-defined architectures, but lacks support for 3D integration as well, posing a significant limitation for exploring vertically integrated designs.

Limitations and LaZagna Capability. Despite notable progress in 3D FPGA research, several key limitations remain. Most prior studies are constrained to fixed or narrowly defined architectural templates, hindering systematic and generalizable design space exploration. Many omit critical components such as DSPs and BRAMs, which are essential in modern FPGA applications. Furthermore, existing evaluations are predominantly simulation-based and lack support for generating synthesizable RTL or programming bitstreams, making full end-to-end validation infeasible. No unified framework exists for consistent, apples-to-apples comparisons across different 3D integration strategies, and important architectural dimensions, such as via placement, layer heterogeneity, and switch block topology, have not been explored in a cohesive manner.

In contrast, LaZagna fills this gap by providing the first fully automated, open-source framework for comprehensive 3D FPGA architecture genera-

tion and evaluation. It unifies architectural modeling, RTL and bitstream generation, and performance evaluation into a scalable infrastructure, enabling systematic exploration across a broad range of 3D design spaces.

III. TOOL WORKFLOW

Figure 1 illustrates the overall workflow of LaZagna, which consists of two distinct flows: one for generating the FPGA fabric and another for generating the bitstream used in benchmark evaluation.

1. Fabric Generation Flow: This flow begins with a user-defined description of the target 3D FPGA architecture, specified via configurable input parameters. Based on this description, LaZagna generates synthesizable RTL using a customized version of OpenFPGA with extended support for 3D integration. The RTL can then be passed through standard physical design steps, including placement and routing (PnR)—a process we refer to as **fabric PnR**—to produce a GDSII layout ready for fabrication, and accurate PPA metrics.

2. Bitstream Generation Flow: In this flow, the user provides benchmark circuits to evaluate the performance of the generated 3D FPGA. These circuits are mapped onto the architecture using VTR, in a process we call **benchmark PnR**, to clearly distinguish it from fabric PnR. After benchmark PnR, key performance metrics such as WL and CPD are extracted. The routing results are then passed to our customized 3D-enabled OpenFPGA to generate a programming bitstream, which is used to configure the 3D FPGA.

A. 3D FPGA Architectural Parameters

LaZagna accepts several architectural parameters that allow users to define custom 3D FPGA configurations, as illustrated in Fig. 1(a). Table II summarizes all supported parameters, including both standard options and new configurations introduced by LaZagna. Fig. 2 visualizes some parameters. This section details the key parameters supported by LaZagna.

1) Vertical Connection Types (Fig. 2a)

Vertical connections are one of the most critical considerations in 3D FPGA design, as they determine how layers are interconnected and directly impact WL, CPD, and vertical via count and density. As shown

FPGA Parameters	Options
(1) Vertical Connection Types (Fig. 2a)	3D CB, 3D CB-O, 3D SB, 3D Hybrid, 3D Hybrid-O (can be easily extended)
(2) 3D SB Percentage and Locations (Fig. 2b)	0 to 100% with placement pattern: [Perimeter, Center, Random, Repeated Interval, Custom]
(3) 3D Switch Block Patterns (Fig. 2c)	Any two 4-integer combinations, e.g., Input [0,1,2,3], Output [1,2,3,4]
(4) Vertical Connection Delay Ratio (Fig. 2d)	A floating value describing the delay ratio between horizontal and vertical connections
(5) Layer Count and Heterogeneity (Fig. 2e)	Arbitrary layer count; layers can be: Homo, Non-Logic Hetero, Logic Hetero
(6) Normal FPGA Parameters	Channel Width, Grid Size, LUT Size, Segment Length, etc.

TABLE II: Supported 3D FPGA architectural parameters.

in Table I, previous studies explore only a limited subset of these options. LaZagna not only supports all those types but also introduces new, highly configurable vertical interconnect schemes.

We categorize vertical connection types into the following classes, visualized in Fig. 2a. For clarity, only connections from Layer 2 to Layer 1 are shown; the reverse direction is omitted but implied.

- **3D CB:** All input and output pins of grid elements (including CLBs, IOs, DSPs, and BRAMs) are allowed to connect across layers via vertical vias. This configuration enables layer crossings at both the source and destination of a net and is natively supported by VTR. As shown in Fig. 2a, CBs in Layer 2 can connect to CLBs in Layer 1, and vice versa. While this approach offers maximum routing flexibility, it incurs a high vertical via count.
- **3D CB-O:** To reduce via usage, this variant restricts cross-layer connectivity to only the output pins of grid elements. Thus, nets only traverse layers at their source. This is also supported by VTR.
- **3D SB:** Cross-layer connections are confined to switch blocks. Nets may change layers only when encountering a designated 3D SB. This model localizes vertical vias to routing junctions, potentially simplifying integration and enabling more controlled via placement.
- **3D Hybrid:** This configuration combines the benefits of both 3D CB and 3D SB designs by allowing cross-layer input/output pin connections and SB-based crossings. It maximizes routing flexibility and vertical utilization but leads to a higher via count.
- **3D Hybrid-O:** A variant of 3D Hybrid that restricts cross-layer connections to output pins only. It provides greater flexibility than 3D CB-O or 3D SB alone while reducing vertical via demand compared to the full 3D Hybrid model.
- **Others (User-Defined):** Beyond the predefined types, LaZagna allows users to define custom vertical connection strategies. For instance, *3D CB-I* or *3D Hybrid-I* can be specified to enable only input pin crossings. Users may also configure partial inter-layer connectivity, such as selecting specific pins or SB tracks for vertical connections. This fine-grained control enables exploration of a significantly broader and more realistic 3D design space.

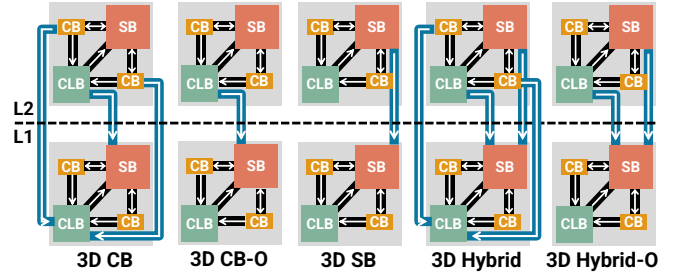
Each vertical connection type presents trade-offs between routing flexibility and vertical interconnect overhead, making it a critical design knob for future 3D FPGA architecture DSE. We will conduct a detailed case study comparing these connection types in Sec. IV-C1.

2) 3D SB Placement Locations (Fig. 2b)

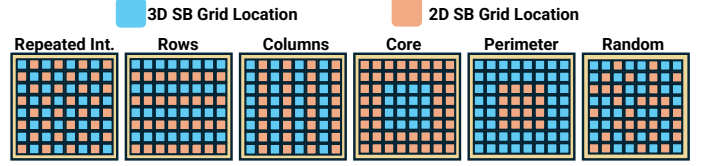
LaZagna allows users to configure both the percentage and spatial distribution of 3D SBs across the FPGA grid. Several default placement patterns are supported, including: **Repeated Interval**, **Rows**, **Columns**, **Core**, **Perimeter**, and **Random**. Users may also define custom placement strategies. For each pattern, users can specify the desired percentage of SBs to be 3D, and LaZagna automatically calculates the corresponding number and distributes them accordingly across the fabric. Additionally, users may provide an explicit list of 3D SB locations via a CSV file, allowing full manual control over 3D SB placement. The impact of various 3D SB placement strategies will be evaluated through a dedicated case study in Section IV-C2.

3) 3D Switch Block Connection Patterns (Fig. 2c)

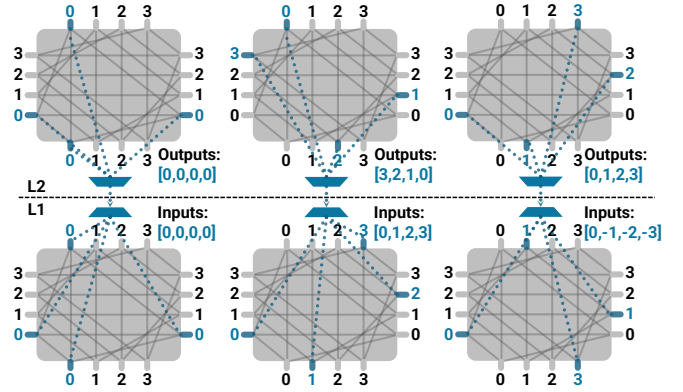
To extend a traditional 2D SB to 3D, vertical cross-layer routing tracks are added. While a 2D SB connects tracks from four planar sides—left,



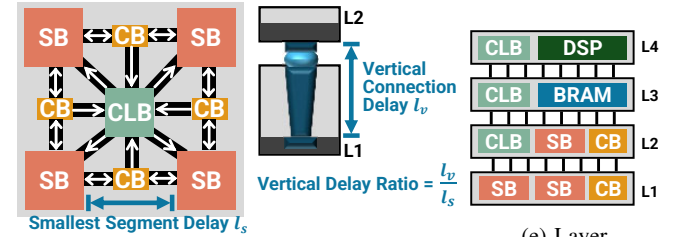
(a) Supported different vertical connection types.



(b) Examples of different 3D SB placement strategies.



(c) Examples of different 3D SB connection patterns.



(d) Vertical delay ratio.

(e) Layer heterogeneity.

Fig. 2: LaZagna input parameters.

bottom, right, and top—a 3D SB can support up to six sides by including vertical connections to layers above and below.

As in 2D FPGAs, the routing behavior of a switch block is governed by a connection pattern that defines how tracks from different sides are interconnected. In the 3D context, this pattern must also account for vertical routing. Two types of patterns must be specified: the output pattern, which determines which planar tracks drive each cross-layer output track, and the input pattern, which determines how each cross-layer input track connects to planar tracks.

In LaZagna, these patterns are user-defined. Both the input and output patterns are expressed as a sequence of four integers corresponding to the four planar sides of the switch block in counter-clockwise order. Each integer indicates the starting track index on that side to be used in vertical connectivity. Consider a switch block with channel width W and an output pattern defined as $[i_0, i_1, i_2, i_3]$. The k -th output cross-layer track will be driven by the planar tracks indexed as $(i_0 + k) \bmod W$ on the left side, $(i_1 + k) \bmod W$ on the bottom, $(i_2 + k) \bmod W$ on the

right, and $(i_3 + k) \bmod W$ on the top, where k ranges from 0 to $W - 1$. The input pattern operates analogously, except that the specified planar track indices are the targets driven by each vertical input track.

Fig. 2c presents a variety of input and output patterns and visualizes how they are realized within the 3D SB structure. In FPGA architectures with segment lengths greater than one, it is possible for the number of input tracks to exceed the number of output tracks. In such cases, the defined pattern is repeated until all input tracks are assigned an output.

4) Vertical Connection Delay Ratio (Fig. 2d)

This parameter specifies the delay cost of vertical interconnects relative to horizontal connections. Since LaZagna is designed to be technology-independent, users can define a delay ratio that scales the vertical connection delay with respect to a baseline, like the delay of a driver switch on the fabric. Alternatively, users may specify the absolute delay value for vertical interconnects if desired. This abstraction enables exploration of different vertical delays without relying on a technology node.

5) Layer Count and Heterogeneity (Fig. 2e)

This parameter allows users to specify both the number of layers in the 3D FPGA and the type of blocks, such as logic elements and connection components, assigned to each layer. As discussed in Section II, LaZagna supports three types of layer configurations: **Homogeneous, Non-Logic Heterogeneous**, and **Logic Heterogeneous**. For example, for a 3-layer logic heterogeneous architecture, a user may choose to place DSP blocks only on the top layer to improve thermal dissipation, allocate additional BRAM blocks to a middle layer for memory balancing, and concentrate routing elements such as SBs and CBs on the bottom layer.

6) All Standard 2D FPGA Parameters

LaZagna also supports all conventional parameters used in 2D FPGA architectures, as provided by VTR and OpenFPGA. These include grid size, channel width, LUT size, inclusion and configuration of hard IP blocks (e.g., DSPs and BRAMs), and others.

Vast design space. With all these configurable parameters and support for easy future extensions, LaZagna enables exploration of a vast and flexible design space for 3D FPGAs. For example, even when 2D FPGA parameters are fixed, only two homogeneous layers are used, the input/output pattern indices for 3D SBs are restricted to the range of -3 to 3, and only the default 3D SB placement patterns are considered, there are still **1,729,440,302 unique 3D FPGA configurations** possible by changing the Connection Type and SB placement, and pattern. All of these configurations can be placed and routed, and have corresponding RTL and bitstreams generated using LaZagna. When custom 3D SB placements are also included, the design space grows exponentially—exceeding 2^{130} possible configurations—even without considering any 2D FPGA parameter variations. While many of these configurations may share similar characteristics, this immense design space highlights the potential of LaZagna to drive the exploration of future 3D FPGA architectures.

B. 3D Fabric Routing Resource Graph Creation

Once the 3D FPGA architectural parameters are specified and parsed, LaZagna constructs a base Routing Resource Graph (RRG) (Fig. 1b) and extends it to a 3D-RRG (Fig. 1c). This RRG is essential for both fabric RTL generation and bitstream creation.

Base RRG. The base RRG represents the FPGA's interconnect structure and is utilized by VTR during benchmark placement and routing (PnR). It serves two key roles. First, it allows benchmark PnR to be formulated as an optimization problem that aims to minimize the critical path delay (CPD) while satisfying architectural and placement constraints. Second, because the RRG encodes the complete connectivity of the fabric, it is also used during RTL generation in OpenFPGA to classify each node and edge by its corresponding architectural block (e.g., CLB, SB, etc.).

As shown in Fig. 1(b), the RRG includes six standard node types defined by VTR: IPIN, OPIN, SOURCE, SINK, CHANX, and CHANY. IPIN and OPIN (labeled 'I' and 'O' in the figure) represent the input and output pins of logic blocks at each grid location. SOURCE and SINK (not shown in the figure) interface between the routing network and internal logic, where each SINK connects to an IPIN, and each SOURCE

connects to an OPIN. CHANX and CHANY ('CX' and 'CY') represent horizontal and vertical routing tracks, respectively, within the same layer.

For 3D architectures that do not use 3D switch blocks, for example 3D CB and 3D CB-O, the base RRG already includes all required connectivity, as these designs are natively supported by VTR.

3D-RRG. To support architectures with custom 3D SB designs, LaZagna extends the base RRG to a 3D-RRG by introducing additional nodes and edges, as illustrated in Fig. 1(c).

As discussed in Section III-A3, in a physical 3D SB, each input track on the source layer is selected via a multiplexer based on the input pattern. The selected signal is then transmitted to the sink layer and distributed to output tracks according to the output pattern.

To represent this in the 3D-RRG, LaZagna introduces a new node type: CHANZ ('CZ' in figure), which models vertical routing. For each inter-layer SB connection, one CHANZ node is created on the source layer and one on the sink layer. The source-layer CHANZ node connects input tracks to the sink-layer CHANZ node, based on the SB's input pattern. The sink-layer node then fans out the signal to output tracks according to the output pattern.

To describe the direction of vertical connections, LaZagna extends VTR's standard routing directions, Incrementing and Decrementing for planar routing, and introduces four new vertical directions: **Above Incrementing**, **Above Decrementing**, **Under Incrementing**, and **Under Decrementing**. *Above* or *Under* denotes the side of the layer the connection is on. *incrementing* or *decrementing* refers to the track's direction between the layers. *Incrementing* means the signal is going from a lower layer to a higher one, *decrementing* means the opposite.

For example, a connection labeled *Above Incrementing* indicates that the signal travels upward (i.e., towards a higher layer index), this implies that it is an output of the current layer. Although these directions are not utilized by VTR's algorithms, they are useful for RTL generation, as they explicitly encode the vertical connectivity structure. This allows LaZagna to efficiently identify source and sink layers for each inter-layer connection during RTL generation.

C. 3D FPGA Architecture RTL Generation

Once the 3D-RRG is constructed, LaZagna modifies the OpenFPGA flow to generate synthesizable RTL for the 3D FPGA architecture. The primary enhancement involves recognizing and handling the new node and edge types introduced in the 3D-RRG.

3D-RRG Annotation. Annotation refers to mapping nodes and edges in the RRG to their corresponding physical components in the FPGA architecture (e.g., CLBs, SBs). To correctly generate a 3D fabric, our modified OpenFPGA annotates all elements of the RRG, including newly added inter-layer connections, with their associated logic or routing blocks. Specifically, LaZagna annotates the following cross-layer signals:

- **Inter-layer Grid Inputs:** Each CB must identify inter-layer routing tracks and connect them to the correct input pins of grid locations. To manage this, each CB performs two tasks: inter-layer output identification and input reception. First, during its creation, each CB analyzes its routing tracks' outgoing edges to identify inter-layer edges and outputs them. In the second step, the CB checks if another CB at the same location on a different layer is sending an inter-layer signal. If such a signal exists, the inter-layer connection is added as a connection to the CB, and is connected to the correct pin's input multiplexer.
- **Inter-layer Grid Outputs:** Each SB is responsible for receiving inter-layer grid output signals and connecting them to the appropriate routing tracks on the SB's layer. To manage this, each SB, during its creation, checks for connections from other layers' grid location outputs. If such a connection exists, it's added to the SB and connected to the respective multiplexers of the routing tracks to which it links.
- **3D SB Signals.** Each 3D SB must also correctly identify inter-layer signals from other 3D SBs. To send an inter-layer signal, the SB multiplexes input routing tracks based on the SB pattern, marking the output as an inter-layer signal. To receive an inter-layer signal, the SB inspects other SBs with the same location but on different layers; if an

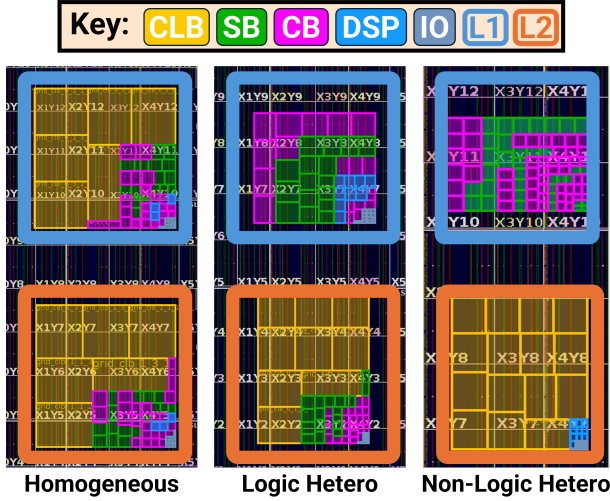


Fig. 3: Implementation of different layer heterogeneity 3D FPGAs onto a Xilinx Virtex Ultrascale+ using RTL generated by LaZagna

inter-layer output is detected, the SB de-multiplexes it based on its output pattern and routes it to the appropriate output tracks.

RTL Generation. Once annotation is complete, RTL generation becomes a deterministic mapping process. A standard cell library (including flip-flops, multiplexers, LUTs, etc.) is provided to OpenFPGA. Using this library, the fabric RTL is constructed by translating the annotated RRG into a complete netlist. Each connection and logic block from the graph is assigned a corresponding RTL primitive, and grouped into higher-level modules.

The generated RTL is organized hierarchically to reflect the layered structure of the fabric. For example, in a two-layer architecture, the output includes files such as `top.v`, `layer1.v`, and `layer2.v`. Each layer file contains only the modules relevant to that layer (e.g., CLBs, SBs). These modules continue to decompose hierarchically down to individual standard cell instances. All cross-layer signals are explicitly defined in the top module, making inter-layer connectivity easy to trace.

Importantly, this RTL generation process is independent of the benchmark PnR. If the 3D-RRG can be created, the RTL can be generated, even for fabrics not currently supported by VTR. This allows LaZagna to produce RTL for logic-heterogeneous 3D architectures and those with more than two layers, which are beyond the capabilities of VTR.

For non-logic heterogeneous architectures, the resulting RTL is structurally similar to that of a 2D FPGA. The only difference lies in module hierarchy: modules are organized by physical layer rather than as a single design. This separation improves design clarity and facilitates physical layout by clearly delineating the blocks assigned to each layer.

D. Benchmark Logic Synthesis, Placement, and Routing

With the 3D-RRG generated, the benchmark circuit undergoes logic synthesis with Yosys (performed independently of 3D-RRG), followed by benchmark PnR onto the 3D fabric using VTR. Since the 3D-RRG reflects the architecture of the target FPGA, VTR can apply its PnR algorithms directly and produce valid routing results. These results include performance metrics such as CPD, WL, and resource utilization. We note that a key limitation of VTR’s 3D placement algorithm is that logic blocks are assigned to specific layers during initial placement and are rarely allowed to move across layers thereafter. This constraint may lead to suboptimal placements. Moreover, VTR does not currently support heterogeneous 3D architectures or CB-based vertical connectivity for architectures with more than two layers. Addressing these limitations remains an avenue for future toolchain enhancements.

E. Bitstream Generation

Following benchmark placement and routing, LaZagna extends OpenFPGA to generate the programming bitstream for the 3D fabric. The

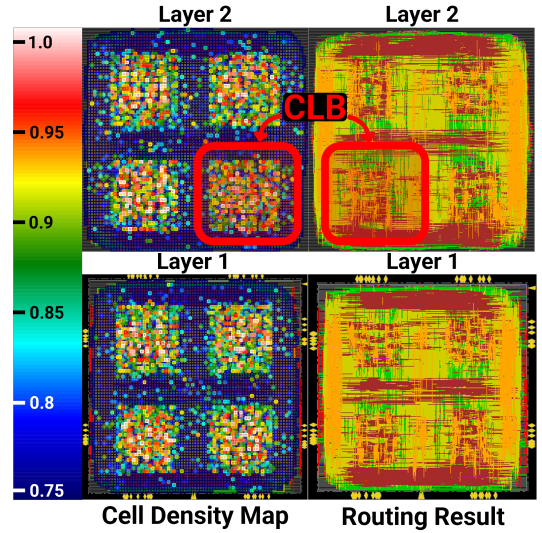


Fig. 4: Physical implementation of a two-layer homogeneous 3D-SB FPGA. Routing closure is achieved, verifying LaZagna’s RTL generation.

process mirrors the RRG annotation procedure described in Section III-C, but instead of annotating all nodes and edges in the RRG, it focuses solely on the routed nodes and edges reported by VTR. As shown in Fig. 1(d) where routed edges are highlighted in red. OpenFPGA traces each signal path, determines the sequence of blocks traversed, and identifies the corresponding configuration bits to activate the routing resources. For 3D grid inputs, this involves configuring CB multiplexers to correctly transmit and receive inter-layer signals. For 3D grid outputs and 3D SBs, the configuration memory bits of the corresponding multiplexers are updated to reflect vertical connections. These enhancements ensure that the generated bitstream correctly configures the 3D FPGA to match the routing determined during benchmark PnR.

F. Tool Output

LaZagna produces three primary outputs: the RTL for the 3D FPGA fabric, benchmark PnR results, and the corresponding bitstream. The RTL enables downstream analysis and physical design, allowing designers detailed power, performance, area, and thermal (PPAT) evaluations, and potentially GDSII layouts for 3D FPGAs.

IV. EXPERIMENTAL RESULTS

This section demonstrates the capabilities of LaZagna in two critical aspects. First, we show that synthesizable RTL for 3D FPGA fabrics can be automatically generated, producing valid bitstreams and maintaining compatibility with industry-standard physical design tools. Second, to highlight the framework’s utility for architectural exploration, we conduct five case studies by varying key 3D FPGA design parameters, such as vertical interconnect types and 3D SB designs, and evaluate key performance metrics. Importantly, this work does not advocate for a specific 3D architecture nor claims superiority over conventional 2D designs, as performance is inherently dependent on packaging and process technologies. Rather, our goal is to enable systematic and reproducible 3D FPGA design space exploration.

A. Experimental Setup

We evaluate the generated 3D FPGA fabrics using a subset of the Koios benchmark suite [14], as also adopted in prior work [10]. Koios provides a representative collection of large-scale, contemporary RTL designs that heavily utilize embedded DSPs and BRAMs—features critical to realistic architectural assessments. We selected a subset of Koios benchmarks that satisfy two criteria: (1) they can be placed and routed on a 2D baseline FPGA fabric within one hour, and (2) they fit within the logic and memory resources of our target grid.

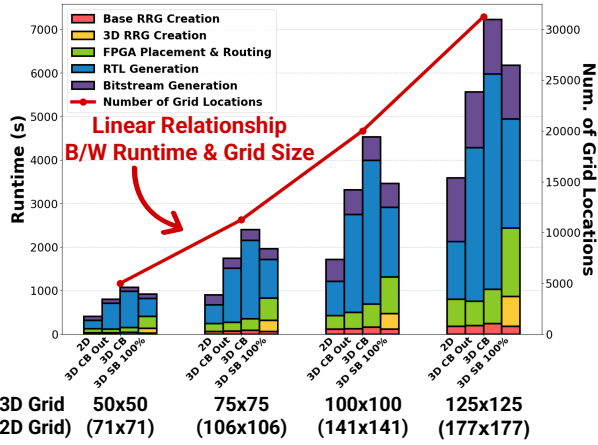


Fig. 5: Runtime breakdown of LaZagna on various Grid Sizes.

All experiments were conducted on a server running Red Hat Enterprise Linux 8.5.0 with a Xeon 6226R processor and 512 GB of RAM. PnR of the benchmarks were done on VTR version 8.1. Fabric RTL and bitstream generation were done using our customized OpenFPGA 1.2 flow.

B. Tool Demonstration

1. RTL and Fabric Generation To demonstrate the RTL generation capabilities of LaZagna, three 2-layered 3D FPGA fabrics with varying layer heterogeneity were designed. These designs were synthesized and implemented on a Xilinx Ultrascale+ FPGA using Vivado. The resulting floorplans are shown in Figure 3. The types of heterogeneity tested were: homogeneous, where the layers are the same; logic heterogeneous, with DSPs on one layer and CLB on the other; and non-logic heterogeneous, with SBs and CBs on one layer, and DSPs and CLB on the other.

To verify both the functionality of the generated 3D FPGA RTL and the validity of the bitstream generated, the s298 MCNC benchmark [15] was implemented onto the homogeneous and non-logic heterogeneous fabrics. Bitstream generation is not currently possible for the logic heterogeneous architectures, as explained in Section III-D. The bitstream was generated using the flow described in Section III. The fabrics were then functionally verified by programming them with the bitstream in RTL simulation and comparing their outputs to the expected results. This process confirmed the correctness of both the generated RTL and bitstream.

2. Physical Implementation To validate the fabric PnR feasibility of the generated RTL, we physically implement a 4x4 homogeneous 3D-SB FPGA architecture using Cadence® Genus and Innovus. The implementation leverages the open source FreePDK45 process [16] with a modified Routing Technology Kit (RTK) LEF, upgraded to version 5.8 with face-to-face stacking, to enable compatibility with the Innovus 3D flow. The congestion map and routing result of the fabric are shown in Fig.4. This demonstrates that our fabric RTL can be used with commercial physical design tools to achieve accurate PPA and fabricate 3D FPGAs.

3. Tool Runtime Fig. 5 presents the end-to-end runtime of LaZagna, using the `eltwise_layer` benchmark across various FPGA grid sizes. As expected, 3D FPGA configurations incur higher runtime than their 2D counterparts with equivalent grid dimensions. This increase is primarily due to the added complexity of 3D fabrics, which feature a greater number of interconnect edges and logic nodes that must be processed during the tool workflow. Importantly, the runtime exhibits a near-linear relationship with the number of grid locations, demonstrating that LaZagna scales predictably with fabric size, making it feasible to apply the tool to larger architectures without incurring prohibitive overhead.

C. Case Studies

To demonstrate the capabilities of LaZagna, we conducted five case studies to explore the 3D FPGA architectural design space. These studies highlight the tool’s flexibility and effectiveness in enabling comprehensive architectural exploration. Table III summarizes the 3D FPGA parameters

Arch. Parameters	Values & Options	Fixed / Varied
Layer count & Grid Size	2 layers; 100x100 (2D: 141x141)	Fixed
Num. of CLBs	14,308 (2D: 14,317)	Fixed
Num. of DSPs	2,548 (2D: 2,502)	Fixed
Num. of BRAMs	2,352 (2D: 2,502)	Fixed
LUT & Cluster & CW Segment Lengths	LUT: 6; Cluster: 10; CW: 300 4 [260 tracks], 16 [40 tracks]	Fixed
Vertical Connection Types	3D SB	Varied in study (1)
3D SB Percentage	50%	Varied in study (1)
3D SB Placement	Repeated Interval	Varied in study (2)
SB Patterns	Wilton (horizontal), Subset (vertical)	Varied in study (3)
Vertical Delay Ratio	0.739 [10] (137 ps)	Varied in study (4)

TABLE III: FPGA architectural parameters used in case studies.

used across the case studies, indicating which parameters remain fixed and which are varied in each specific study.

Key evaluation metrics for 3D FPGA architectures include the number of required vertical interconnects, total wirelength (WL), and critical path delay (CPD). It is important to note that both WL and CPD are highly sensitive to the quality of PnR results, physical design technology node, and 3D packaging technology. Consequently, the reported WL and CPD values may vary significantly with improved 3D-aware PnR tools or the adoption of advanced packaging solutions.

1) Impact of Vertical Connection Types

In this study, we compare a variety of 3D connection types against a baseline 2D architecture to evaluate the aforementioned metrics. Each architecture is configured to have a comparable number of CLBs, DSPs, and BRAMs, while varying the routing resources.

Specifically, we evaluate the following vertical connection types: 3D CB, 3D CB-O, 3D SB, 3D Hybrid, and 3D Hybrid-O, as defined in Section III-A and illustrated in Fig. 2a. For architectures utilizing 3D SBs, we vary the proportion of SBs with vertical connections to be 100% (all SBs are 3D), 66% (two-thirds), and 33% (one-third).

For the Hybrid configurations, this percentage applies only to the SBs on the grid that incorporate 3D connectivity. In 3D Hybrid architectures, all input and output pins of grid locations have inter-layer connections, and only the output pins are interlayer for 3D Hybrid-O architectures.

Number of vertical connections. 3D FPGA design is fundamentally constrained by the density of available vertical inter-layer connections, which is determined by the chosen integration technology (e.g., TSVs, MIVs, or microbumps). For instance, hybrid bonding offers up to 302 vertical connections per grid location at a 1 μ m pitch, but only 12 connections at a 5 μ m pitch, based on the ASAP7 PDK [10]. MIVs provide a more attractive solution for 3D FPGAs due to their finer pitch, down to 0.1 μ m [17] and reduced interconnect latency. As a result, MIVs can support significantly higher vertical connection densities compared to hybrid bonding or TSVs. However, MIV fabrication requires homogeneous layer stacks, which may limit its applicability in some designs.

Table IV shows the number of vertical connections required for each evaluated architecture. Most architectures exhibit a reasonable number of vertical connections per grid location, remaining below the 302-connection limit achievable with 1 μ m pitch hybrid bonding. Specifically, the 3D CB-O, 3D SB, and 3D Hybrid-O architectures require fewer vertical connections, whereas architectures such as 3D CB and 3D Hybrid demand higher connection densities due to the large number of inter-layer connections at CLB inputs. Future work may explore techniques such as routing channel multiplexing to reduce the number of vertical connections.

WL and CPD. Figure 6 and Table IV summarize the WL and CPD results. All 3D architectures outperform the 2D baseline, achieving CPD reductions of 3.1%–6.7% and WL reductions of 4.2%–22.9%. Among them, the 3D Hybrid architectures deliver the most balanced improvements in both metrics, with the 3D CB architecture achieving the largest WL reduction and the 100% 3D SB configuration yielding the greatest CPD reduction. While WL improvement shows a clear positive correlation with the number of vertical connections, CPD does not follow the same trend: the 3D SB architectures achieve the best CPD reductions despite having relatively few vertical connections per grid.

Takeaways. (1) 3D FPGAs show promising improvements in CPD

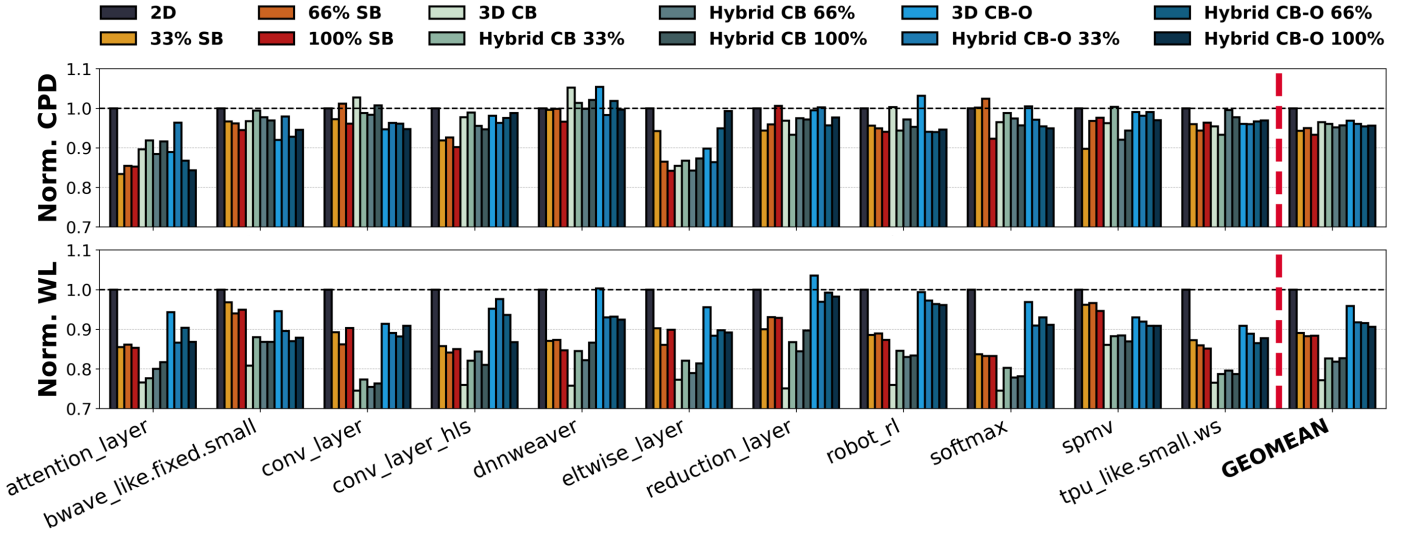


Fig. 6: Case Study (1): CPD and WL results of various vertical connection types, normalized to 2D FPGAs, using Koios Benchmarks.

Vertical Connection Type	# of Vert. Connections	Vert. Conns. Per Grid	WL Geomean (% reduc.)	CPD Geomean (% reduc.)
3D CB	3,038,192	316.3	0.771 (22.9%)	0.965 (3.5%)
3D CB-O	431,984	45.0	0.958 (4.2%)	0.969 (3.1%)
3D SB 100%	750,680	78.2	0.884 (11.6%)	0.933 (6.7%)
3D SB 66%	495,598	51.6	0.882 (11.8%)	0.950 (5.0%)
3D SB 33%	247,832	25.8	0.890 (11.0%)	0.943 (5.7%)
3D Hybrid 100%	3,788,872	394.5	0.827 (17.3%)	0.957 (4.3%)
3D Hybrid 66%	3,533,790	367.9	0.818 (18.2%)	0.951 (4.9%)
3D Hybrid 33%	3,286,064	342.2	0.826 (17.4%)	0.960 (4.0%)
3D Hybrid-O 100%	1,182,664	123.1	0.906 (9.4%)	0.956 (4.4%)
3D Hybrid-O 66%	927,582	96.6	0.916 (8.4%)	0.955 (4.5%)
3D Hybrid-O 33%	679,816	70.8	0.917 (8.3%)	0.960 (4.0%)

TABLE IV: Impact of vertical connection types. Results normalized to 2D FPGA on Koios benchmark subset.

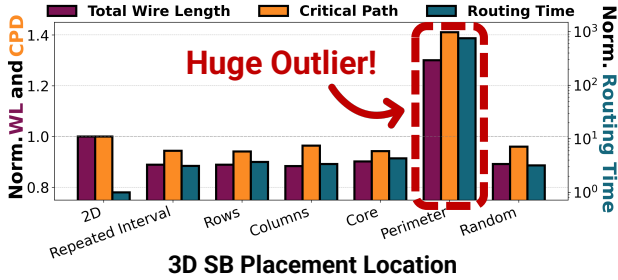


Fig. 7: Case Study (2): CPD and WL of different 3D SB Placements.

and WL compared to 2D FPGAs. (2) While more vertical interconnects improve WL performance, the relationship with CPD is less straightforward, with diminishing returns at higher connection densities. (3) A balanced architecture that combines different vertical connection types tends to yield the most effective trade-offs.

2) Impact of 3D SB Placement

This case study investigates the impact of 3D SB placement on CPD, WL, and routing time. Building on the promising results of the 3D SB connection type from Case Study 1, we explore if the spatial distribution of 3D SBs has a significant effect on performance. We use 3D SB subset connections with 50% 3D SBs and evaluate six placement strategies, illustrated in Fig.2b. The corresponding results are presented in Fig.7. Among the evaluated strategies, the Perimeter placement stands out as a clear outlier, yielding substantially worse CPD and WL than all other approaches and even performing significantly below the 2D baseline. It also incurs an exponentially higher routing time. This degradation is likely due to the VTR placement engine's lack of awareness of vertical

3D SB Pattern Name	Input Pattern	Output Pattern
Subset	[0,0,0,0]	[0,0,0,0]
Off By One Output	[0,0,0,0]	[1,1,1,1]
Revolving Offset	[0,1,2,3]	[0,1,2,3]
Revolving Input	[0,1,2,3]	[0,0,0,0]
Revolving Output	[0,0,0,0]	[0,1,2,3]
Direction Match	[0,1,0,1]	[1,0,1,0]
Symmetric Offset	[-2,-1,1,2]	[0,0,0,0]
Random	[-3,0,2,1]	[3,-1,-2,2]

TABLE V: 3D SB Patterns Tested in Case Study (3).

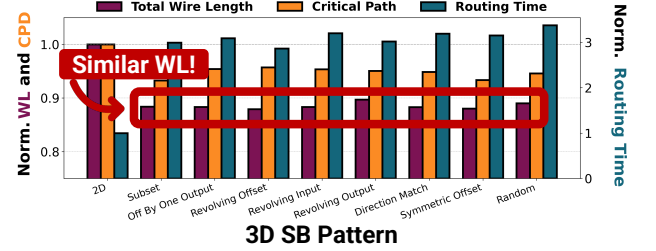


Fig. 8: Case Study (3): CPD and WL of different 3D SB patterns.

connectivity, which may result in logic blocks requiring vertical links being placed far from the perimeter-located 3D SBs, thereby increasing routing congestion and prolonging runtime. These findings underscore the importance of developing 3D-aware placement algorithms.

Excluding the Perimeter case, the remaining strategies exhibit comparable CPD, WL, and routing times, with the Core placement showing a slightly higher runtime than the others. Strategies with more distributed 3D SBs, such as Repeated Interval, Rows, Columns, and Random, achieve the fastest routing times, while those with concentrated 3D SBs, such as Core and Perimeter, perform worse. This suggests that a more uniform distribution of 3D SBs across the fabric can enable more efficient routing.

Takeaways. (1) 3D SB placement significantly impacts routing time, even when the number of 3D SBs is held constant. (2) Uniformly distributing 3D SBs across the fabric enables faster and more efficient routing compared to concentrated configurations. (3) The PnR algorithm used in this study may not be optimized for 3D FPGAs and may yield suboptimal results, highlighting the need for 3D-aware PnR strategies.

3) Impact of 3D SB Connection Patterns

This case study investigates the influence of 3D SB connection patterns on performance. We explored various patterns, outlined in Table V, including revolving and symmetric patterns inspired by the Wilton pattern commonly used in 2D FPGAs [18]. The Wilton pattern, based

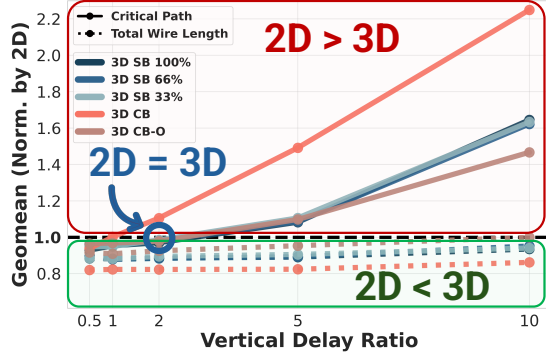


Fig. 9: Case Study (4): CPD and WL under different vertical delay ratios, ranging from 0.5 to 10.

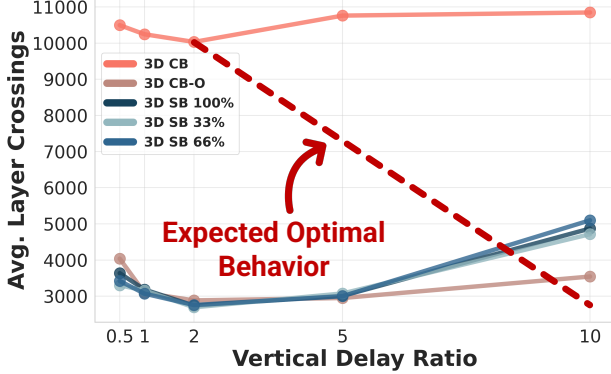


Fig. 10: Case Study (4): Average number of signals that cross layers as vertical delay ratio increases.

on an offset with a modulo operation, enhances routability in 2D by reducing the number of turns required to reach desired tracks. While all vertical connections in 3D inherently involve turns, we investigated whether specific 3D patterns could offer greater benefits compared to others. A randomly generated pattern was also included to show the tool's ability to explore a wide range of patterns.

Results shown in Fig. 8 indicate that WL and CPD are largely unaffected by the SB pattern. CPD and WL show small variation, with CPD improvements ranging from 4.3% to 6.7%, and WL improvements ranging from 10.4% to 12.1%. The symmetric offset and subset patterns yielded the best CPD results. While further investigation is needed to understand the impact of different patterns, these initial results suggest that the placement of 3D SBs has a greater influence than the connection pattern.

Takeaways. (1) 3D SB connection patterns appear to have a minimal effect on WL and CPD. (2) The location of 3D SBs appears to be a more significant factor.

4) Impact of Vertical Connection Delay

This case study examines the effect of vertical connection delay on 3D FPGA performance. Different 3D integration technologies exhibit varying vertical connection densities and speeds. To explore this impact, we varied the vertical delay ratio from $0.5\times$ to $10\times$ the delay of a length-4 horizontal routing segment. As a reference point, [10] used a vertical connection delay of 137ps was used ($0.739\times$ the horizontal segment delay), representing hybrid bonding with a $5\mu\text{m}$ pitch. In our study, a $0.5\times$ vertical delay corresponds to a vertical delay of 93ps, while a $10\times$ ratio corresponds to 1.86ns. This wide range encompasses various technologies, from faster hybrid bonding (93ps or less depending on pitch) to slower TSVs (e.g., 1.5ns as reported in [19]).

Fig. 9 presents the results. CPD exhibits a near-linear relationship with the vertical delay ratio. WL increases slightly for all 3D architectures. However, a key limitation in VTR's placement engine may influence these results. During initial placement, VTR assigns blocks to layers pseudo-randomly, preferring layers containing connected blocks. Subsequently, the algorithm limits inter-layer movement, with only a $\sim 10\%$ probability

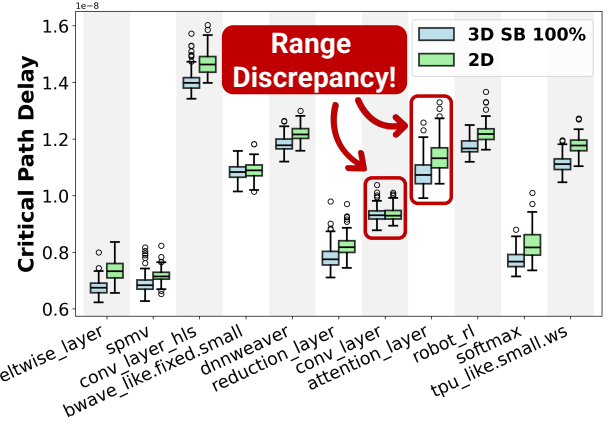


Fig. 11: Case Study (5): Distribution of CPD values after 150 random seed placements on 2D and 3D FPGAs with 100% 3D SBs.

of layer reassignment per block during the placement flow.

This limited layer reassignment potentially limits performance gains, especially considering the interplay between vertical delay and layer crossings. As shown in Fig. 10, the number of layer crossings remains constant for 3D CB-O architectures and decreases slightly for 3D SB and 3D CB architectures as the vertical delay ratio increases. This is a direct consequence of VTR's placement limitations. Ideally, with a low vertical delay, designs should maximize layer crossings to exploit vertical connectivity. Conversely, a high vertical delay should minimize crossings. VTR's limited layer reassignment prevents such dynamic optimization.

Takeaways. (1) CPD increases approximately linearly with vertical connection delay. However, VTR's placement limitations may influence this relationship. (2) 3D FPGAs demonstrate performance comparable to their 2D counterparts when the vertical connection delay is approximately two times that of a horizontal connection. This suggests that even with moderately high vertical delays, 3D architectures can remain competitive.

5) Impact of Benchmark Placement Algorithm

To assess the variability of the benchmark PnR process for both 2D and 3D architectures, a study was conducted using randomized placement seeds. For each architecture tested (2D and 100% 3D SB), 150 unique seeds were generated and used by the placement engine during simulated annealing. The resulting CPD values are presented in Fig. 11.

The results indicate that 3D SB architectures consistently achieve better CPD across all benchmarks. However, the spread of CPD values across different placement seeds is similar for both 2D and 3D architectures. For example, the `attention_layer` benchmark shows a wide variation in CPD depending on the seed, whereas the `conv_layer` benchmark exhibits a much tighter distribution. This suggests that introducing 3D interconnects does not inherently increase the variation in placement and routing outcomes. Nonetheless, the benchmark-specific differences in CPD variability merit further investigation.

V. CONCLUSION AND FUTURE WORK

In this work, we introduced LaZagna, an automated tool that streamlines the design and exploration of 3D FPGA architectures. By generating custom 3D FPGA fabrics based on user-defined parameters, LaZagna facilitates efficient exploration of the vast 3D FPGA design space. And allows the exploration of 3D FPGAs beyond simulation by producing synthesizable RTL that can be physically designed. Our results demonstrate that 3D FPGAs can achieve improvements of up to 6.7% for critical path delay and up to 22.9% for total wire length compared to 2D FPGAs.

Looking ahead, we aim to obtain accurate Power, Performance, Area, and Thermal (PPAT) metrics for 3D FPGAs through physical design implementation. Additionally, we plan to investigate placement and routing algorithms specifically tailored for 3D FPGAs, with the potential to further enhance performance.

REFERENCES

- [1] L. Bamberg, A. Garcia-Ortiz, L. Zhu, S. Pentapati, D. E. Shim, and S. Kyu Lim, "Macro-3D: A physical design methodology for face-to-face-stacked heterogeneous 3D ICs," in *2020 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, Mar. 2020.
- [2] D. K. Nayak, S. Banna, S. K. Samal, and S. K. Lim, "Power, performance, and cost comparisons of monolithic 3D ICs and TSV-based 3D ICs," in *2015 IEEE SOI-3D-Subthreshold Microelectronics Technology Unified Conference (S3S)*. IEEE, Oct. 2015.
- [3] J. Kim, L. Zhu, H. M. Torun, M. Swaminathan, and S. K. Lim, "Micro-bumping, hybrid bonding, or monolithic? a PPA study for heterogeneous 3D IC options," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, Dec. 2021.
- [4] O. Turkyilmaz, G. Cibrario, O. Rozeau, P. Batude, and F. Clermidy, "3D FPGA using high-density interconnect monolithic integration," in *2014 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, Mar. 2014, pp. 1–4.
- [5] M. Lin, A. El Gamal, Y.-C. Lu, and S. Wong, "Performance benefits of monolithically stacked 3D-FPGA," in *Proceedings of the 2006 ACM/SIGDA 14th international symposium on Field programmable gate arrays*. New York, NY, USA: ACM, Feb. 2006.
- [6] W. Faaiq, Z. Jiahao, L. Anni, H. Zifan, C. Jason, and Y. Shimeng, "Monolithic 3D FPGAs utilizing back-end-of-line configuration memories," *arXiv [cs.ET]*, Jan. 2025.
- [7] R. Le, S. Reda, and R. I. Bahar, "High-performance, cost-effective heterogeneous 3D FPGA architectures," in *Proceedings of the 19th ACM Great Lakes symposium on VLSI*, ser. GLSVLSI '09. New York, NY, USA: Association for Computing Machinery, May 2009, pp. 251–256.
- [8] M. Amagasaki, Q. Zhao, M. Iida, M. Kuga, and T. Sueyoshi, "A 3D FPGA architecture to realize simple die stacking," *IPSJ Transactions on System and LSI Design Methodology*, vol. 8, pp. 116–122, 2015.
- [9] M. Amagasaki, M. Ikebe, Q. Zhao, M. Iida, and T. Sueyoshi, "Three dimensional FPGA architecture with fewer TSVs," *IEICE Trans. Inf. Syst.*, vol. 101-D, pp. 278–287, Feb. 2018.
- [10] A. Boutros, F. Mahmoudi, A. Mohaghegh, S. More, and V. Betz, "Into the third dimension: Architecture exploration tools for 3D reconfigurable acceleration devices," in *2023 International Conference on Field Programmable Technology (ICFPT)*. IEEE, Dec. 2023, pp. 198–208.
- [11] M. Lin, A. El Gamal, Y.-C. Lu, and S. Wong, "Performance benefits of monolithically stacked 3-D FPGA," *IEEE Trans. Comput.-aided Des. Integr. Circuits Syst.*, vol. 26, no. 2, pp. 216–229, Feb. 2007.
- [12] K. E. Murray, O. Petelin, S. Zhong, J. M. Wang, M. Eldafrawy, J.-P. Legault, E. Sha, A. G. Graham, J. Wu, M. J. P. Walker, H. Zeng, P. Patros, J. Luu, K. B. Kent, and V. Betz, "VTR 8: High-performance CAD and customizable FPGA architecture modelling," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 13, no. 2, pp. 1–55, Jun. 2020.
- [13] X. Tang, E. Giacomini, B. Chauviere, A. Alacchi, and P.-E. Gaillardon, "OpenFPGA: An open-source framework for agile prototyping customizable FPGAs," *IEEE Micro*, vol. 40, no. 4, pp. 41–48, Jul. 2020.
- [14] A. Arora, A. Boutros, D. Rauch, A. Rajen, A. Borda, S. A. Damghani, S. Mehta, S. Kate, P. Patel, K. B. Kent *et al.*, "Koios: A deep learning benchmark suite for fpga architecture and cad research," in *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 2021, pp. 355–362.
- [15] S. Yang, *Logic Synthesis and Optimization Benchmarks User Guide: Version 3.0*. Microelectronics Center of North Carolina (MCNC), 1991.
- [16] J. E. Stine, I. Castellanos, M. Wood, J. Henson, F. Love, W. R. Davis, P. D. Franzon, M. Bucher, S. Basavarajaiah, J. Oh, and R. Jenkal, "FreePDK: An open-source variation-aware design kit," in *2007 IEEE International Conference on Microelectronic Systems Education (MSE'07)*. IEEE, Jun. 2007.
- [17] S. K. Samal, D. Nayak, M. Ichihashi, S. Banna, and S. K. Lim, "Monolithic 3D IC vs. TSV-based 3D IC in 14nm FinFET technology," in *2016 IEEE SOI-3D-Subthreshold Microelectronics Technology Unified Conference (S3S)*. IEEE, Oct. 2016.
- [18] S. J. E. Wilton, "Architectures and algorithms for field-programmable gate arrays with embedded memory," 1997.
- [19] S.-Y. Huang, Y.-H. Lin, K.-H. Tsai, W.-T. Cheng, S. Sunter, Y.-F. Chou, and D.-M. Kwai, "Small delay testing for TSVs in 3-D ICs," in *DAC Design Automation Conference 2012*. IEEE, Jun. 2012, pp. 1031–1036.